

Data Structures

Complexity Analysis



2018/2019

Review

- Big-O Notation:
 - Used to indicate the type of change of a function:
 - The (N) - Linear
 - $O(N^2)$ - Quadratic
 - ...
 - Useful to analyze and compare the behavior of algorithms without the need to precisely calculate the runtime.



C

- **(Big-O)** $T(N)$ is $O(f(N))$ if there are positive constants c and N_0 such that $T(N) \leq c f(N)$ for $N \geq N_0$.
- Example: is $N^2 + N$ $O(N^2)$?
 - Assume one value for c or N_0 , and try to compute the other so that the inequality is verified.

$$T(N) = N^2 + N; f(N) = N^2$$

- For example, assuming $c=2$, then

$$N^2 + N \leq cN^2 \Leftrightarrow N^2 + N \leq 2N^2 \Leftrightarrow N + 1 \leq 2N \Leftrightarrow N \geq 1$$

Therefore $N_0=1$ (or higher).



Big-O: Definition

- **(Big-Omega)** $T(N)$ It is $\Omega(F(n))$ if there are positive constants c and N_0 such that $T(N) \geq cF(N)$ for $N \geq N_0$.
- Example: is $N \log N \Omega(N)$?



Big-O: Definition

- **(Theta-Grande)** $T(N)$ It is $\Theta(F(n))$ if $T(N)$ is $\Omega(F(n))$ and $T(N)$ is $O(F(n))$.

- Example: is $\frac{N^2}{N-10} \Theta(N)$?



Big-O: Definition

- **(Little-o)** $T(N)$ It is $o(F(n))$ if $T(N)$ is $O(F(n))$ and $T(N)$ is not $\Theta(F(n))$.
- Example: is $N^2 \log N$ $o(N^3)$?



Exercise

- Consider that an algorithm has complexity of order $O(N^2)$. How much does the run time increase if the input size increase 10 times?
- Knowing $T(n) = wN^2$ then $T(10N) = 100wN^2$
 $T(10N) = 100 T(N)$
 - What if $T(n)$ is $O(N)$?
 - What if $T(n)$ is $O(N^3)$?
 - What if $T(n)$ is $O(N \log N)$?



logarithms

- The logarithm (base 2) indicates
 - The number of digits (bits) of a number
 - The number of times that one must be doubled to achieve N
 - The number of times N must be divided (by 2) to reach 1.
 - If an algorithm takes $O(1)$ to reduce the scale of a problem on a given percentage (not necessarily 50%), it will be $O(\log N)$
 - For example, an algorithm at each step removes 10% of the analyzed data until only one value.

$$O(\log_B N) = O(\log N)$$

because?



Search

- Given a number X and the array, return the position of X in A or an indication that X does not exist.
 - Sequential search
 - Binary search
 - Interpolated search



Sequential search

- What is the complexity of:
 - A search without success?
 - A successful search in the worst case?
 - A successful search, the “average” case?



Binary search

- If the search space is ordered, look up the value at the midpoint of the array, and not at the beginning.
 - The number of values that need to be searched is reduced to half (approximately) each iteration.
 - And if we look for the point to $\frac{3}{4}$ of the array, instead of examining the midpoint?
- What is the complexity of a failed search?
- And a successful search, in the worst case?
- And a successful search, the average case?



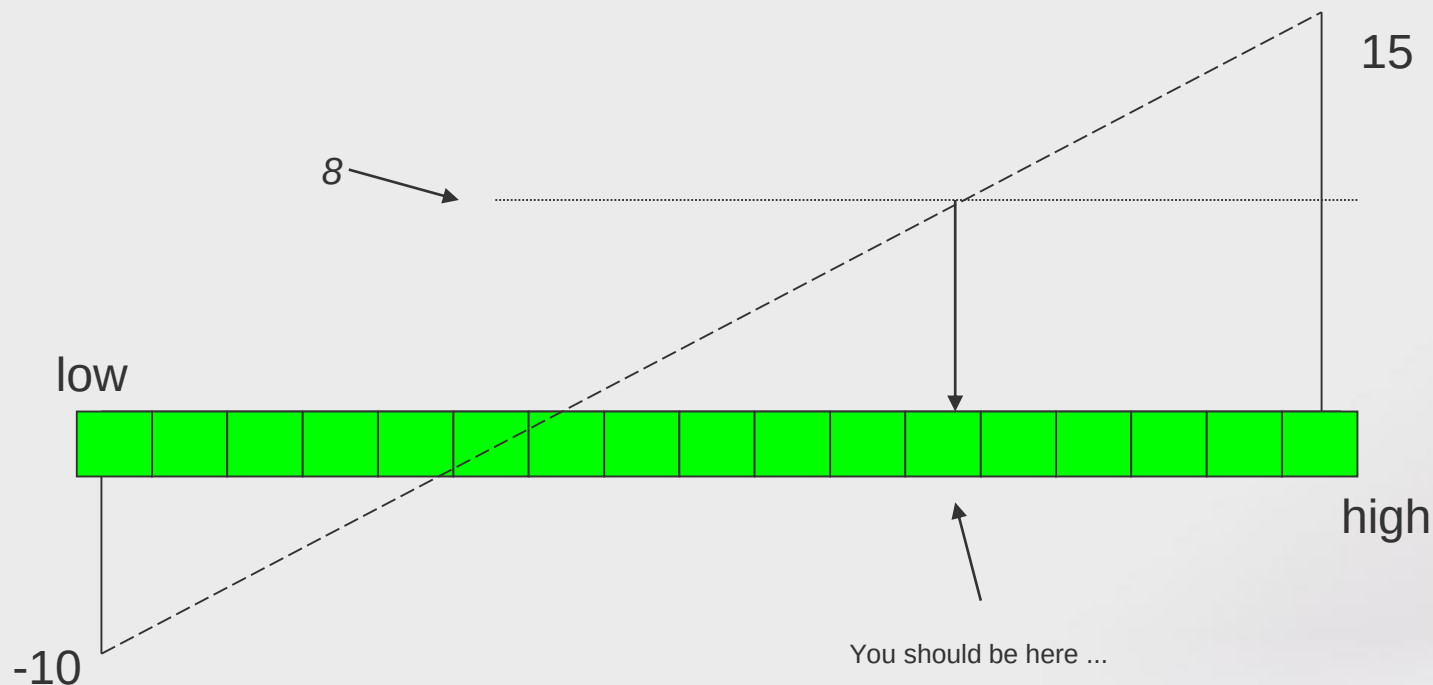
Interpolated search

- If the array is ordered and the distribution is uniform we can do *interpolated search*...
 - Advantageous only if access to data is too costly.
- It is based on interpolating the position of the element sought from the values of the major and minor element.



Interpolated search

- Find the number 8



Interpolated search

- What is the worst case?
 - If the values are not evenly distributed, all the elements are likely to be searched ...
 - (N)
- What is the average case?
 - If the distribution is relatively uniform:
 - $O(\log \log N)$



comparative analysis

- We will study a problem for which many different algorithms exist. Let's review:
 - An exhaustive algorithm, simple but inefficient.
 - A simple improvement of the previous algorithm.
 - An efficient algorithm, but less obvious.



Maximum Contiguous Sequence

- Given a set of integers (possibly negative), determine which is the contiguous sequence (including the empty set) with the largest sum.
- Example:
 - $MSC(\{2, 3, 4, -2, 1\}) = 9$
 - $MSC(\{-2, 1\}) = 1$
 - $MSC(\{-1, -2, -3\}) = 0$
 - $MSC(\{\}) = 0$



MSC - Version 1 (Exhaustive Search)

```
1. public static int maxSeqCont (int [] m)
2. {
3.     maxSoma int = 0;
4.     Int N = m.length;
5.     for (int i = 0; i <N i ++)
6.         for (j = i; j <N; j ++) {
7.             estaSoma int = 0;
8.             for (i = k; k <j; k ++)
9.                 estaSoma + = m [k];
10.            if (estaSoma> maxSoma)
11.                maxSoma = estaSoma;
12.        }
13. maxSoma return;
14. }
```

What is the complexity of this algorithm?



MSC - Version 1

- The dominant factor in this algorithm is the time spent on line 9.
- A rough calculation:
 - The outer loop (i) has N iterations.
 - The intermediate ring (j) is between 1 and $N-1$ iterations
 - The inner loop (k) is performed between 1 and $N-1$ times.
- Multiplying the number of iterations of the various chained cycles, we can assume that the complexity of the algorithm should be $O(N^3)$
 - *This hypothesis can be confirmed by a more accurate analysis.*



MSC – Versão 1

- The exact number of time line 9 is executed is...

$$\sum_{i=1}^N \sum_{j=i+1}^N \sum_{k=i}^j 1 = \frac{N(N+1)(N+2)}{6}$$

- This confirms cubic behavior.
 - Regardless of the time the line takes to execute, it will end up dominating the running time for high values of N .



MSC – Versão 2

- Cubic algorithms are unfeasible in practice
- The three nested loops result in cubic order...
- Is it possible to remove one loop?



MSC – Versão 2

- We're performing unnecessary computations:
 - The inner loop computes $\sum_{k=i}^j A_k$
 - But knowing: $\sum_{k=i}^j A_k = A_j + \sum_{k=i}^{j-1} A_k$
 - Then we can just add A_j to previous iterations' value.
 - Thereby eliminating the inner loop.



MSC – Versão 2

```
1. public static int maxSeqCont(int [] m)
2. {
3.     int maxSoma=0;
4.     int N = m.length;
5.     for (int i = 0; i <N; i ++) {
6.         estaSoma int = 0;
7.         for (j = i; j <N; j ++) {
8.             estaSoma + = m [j];
9.             if (estaSoma> maxSoma)
10.                 maxSoma = estaSoma;
11.         }
12.     }
13. }
```



Just add the new
element to the result
previous.



MSC - Version 2

- What is the complexity?
 - a loop was removed, so that the complexity is reduced to $O(N^2)$
- precise calculation:
 - The number of executions of the inner loop of lines (rows 8 and 9) is given by:

$$\sum_{i=1}^N \sum_{j=i+1}^N 1 = \frac{N(N-1)}{2}$$



MSC – Version 3

- Removing an additional loop we might achieve an linear algorithm
- Is it possible?



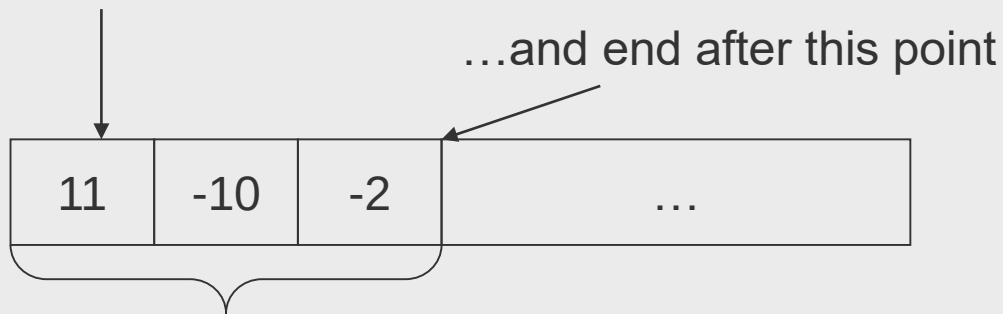
MSC – Versão 3

- If $A_{i,j}$ is a sequence with sum $S_{i,j} < 0$, then $A_{i,q}$, with $q > j$, is not a MSC.
 - Because $S_{j+1,q}$ will be greater than $S_{i,j} \dots$
- Knowing this, the algorithm can be modified to discard subsequences including negative subsums.



- **Example**

MSC cannot start here...



$S = -1$

...because it would be possible to ignore this and obtain a larger sequence.



version 3.0a

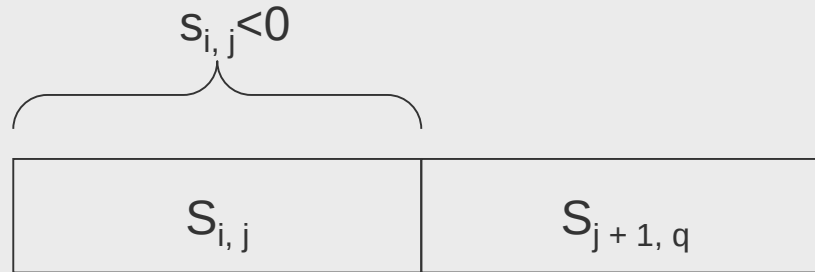
```
1. public static int maxSeqCont (int [] m)
2. {
3.     maxSoma int = 0;
4.     int N = m.length;
5.     for (int i = 0; i <N; i ++) {
6.         estaSoma int = 0;
7.         for (j = i; j <N; j ++) {
8.             estaSoma += m [j];
9.             if (estaSoma <0) ←
10.                break;
11.             if (estaSoma> maxSoma)
12.                 maxSoma = estaSoma;
13.         }
14.     maxSoma return;
15. }
```

You can skip the remaining sequences from this point ...

It is enough?

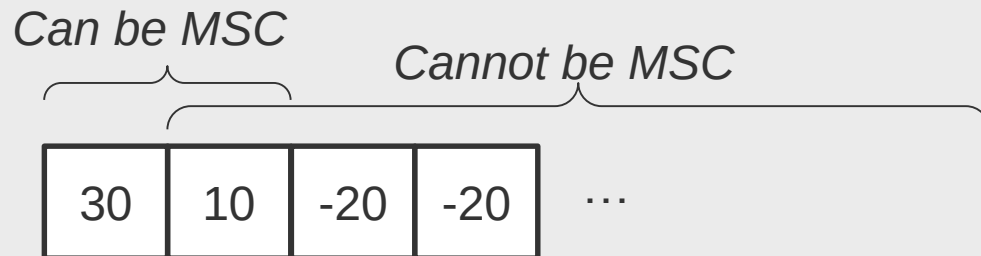


version 3.0b



If $S_{i,j} < 0$, then the maximum was already found or begins after j .
Search can continue after j .

:



Alternative MSC may only exist after this point.



Versão 3.0b

```
1. public static int maxSeqCont(int [] m)
2. {
3.     int maxSoma=0;
4.     int N=m.length;
5.     for(int i=0;i<N;i++){
6.         int estaSoma=0;
7.         for(j=i;j<N;j++){
8.             estaSoma+=m[j];
9.             if(estaSoma<0){ ←
10.                 i=j;
11.                 break;
12.             }
13.             if(estaSoma>maxSoma)
14.                 maxSoma=estaSoma;
15.         }
16.     return maxSoma;
17. }
```

Abandon negative subsequences, and start looking for new MSC from this position



Versão 3.0c

Equivalent code

```
1. public static int maxSeqCont(int [] m)
2. {
3.   int maxSoma=0;
4.   int N=m.length;
5.   int estaSoma=0;
6.   for(int i=0;i<N;i++){
7.     estaSoma+=m[i];
8.     if(estaSoma>maxSoma)
9.       maxSoma=estaSoma;
10.    if(estaSoma<0)
11.      estaSoma=0;
12.  }
13. return maxSoma;
14. }
```

Complexity?



MCS

- We analyzed 3 algorithms:
 - *Exhaustive search* – $O(N^3)$ – *Obvious, easy to understand.*
 - *Simple Improvement* – $O(N^2)$ –
 - *Optimized* – $O(N)$ – *Linear, but harder to understand*
- Performance optimizations reduce clarity.
- Because of this, it is usually necessary to demonstrate optimizations work properly

